

1. Introducción y Arquitectura

1.1 Objetivo

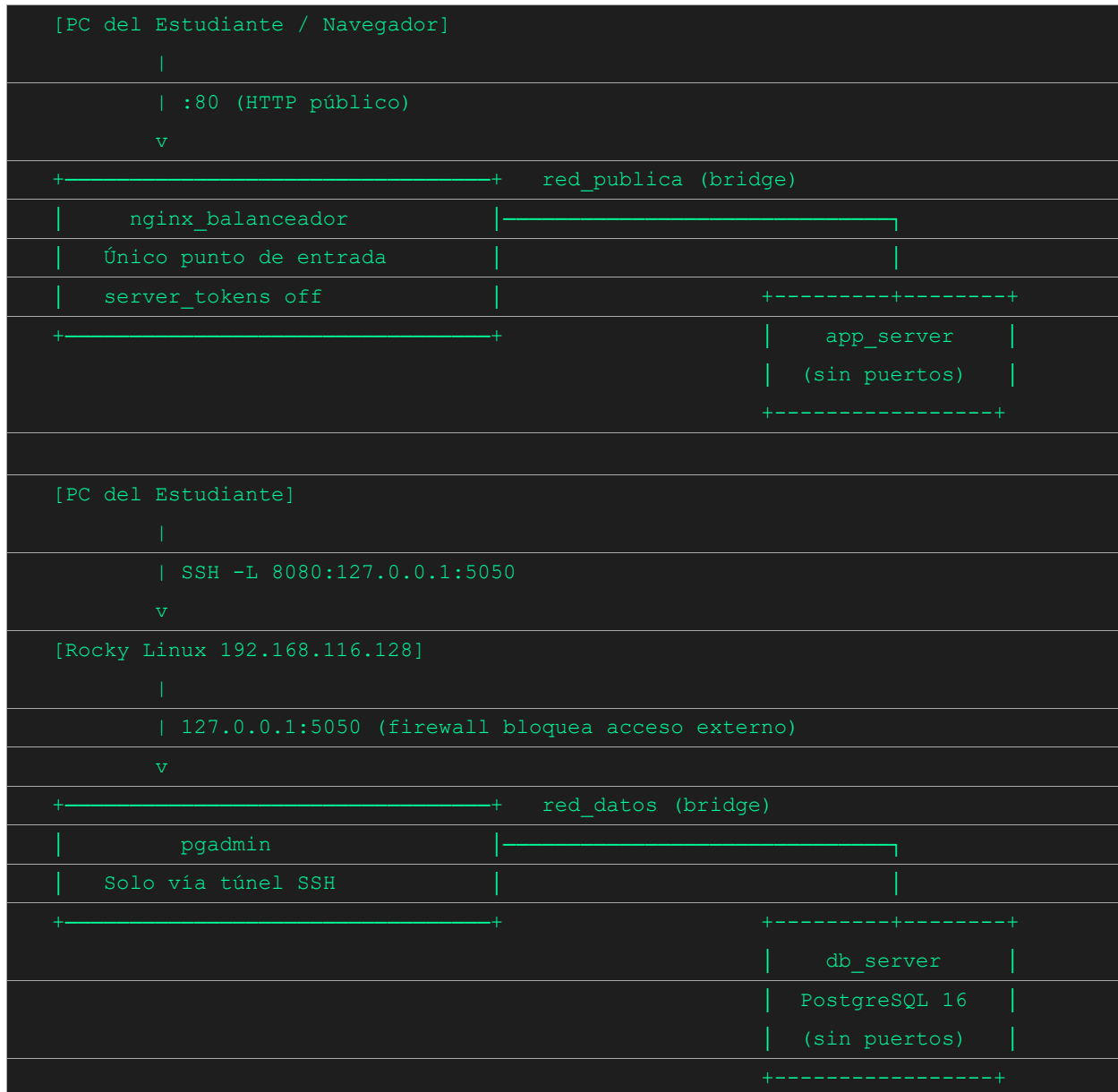
El objetivo de esta práctica es dominar la orquestación de infraestructuras complejas aplicando el enfoque de Infraestructura como Código (IaC). Se diseña un ecosistema coordinado mediante un archivo `docker-compose.yml` que integra cuatro servicios: un balanceador de carga (Nginx), un servidor de aplicaciones interno, una base de datos PostgreSQL con persistencia, y un panel de administración (pgAdmin) accesible exclusivamente mediante túnel SSH cifrado.

Las habilidades desarrolladas incluyen:

- Abstracción de servicios: separación de capa de datos, lógica y gestión.
- Seguridad perimetral: ocultar servicios críticos de la red pública.
- Automatización de dependencias: arranque ordenado con healthchecks.
- Túneles de gestión: acceso cifrado SSH para administrar servicios ocultos.

1.2 Diagrama de Topología

La infraestructura está compuesta por dos redes Docker internas y cuatro contenedores distribuidos como se muestra a continuación:



2. Guía de Uso de los Scripts (Manual de Usuario)

2.1 Requisitos Previos

1. Rocky Linux 9/10 con Docker CE instalado y servicio activo (systemctl status docker).
2. Docker Compose Plugin instalado (docker compose version).
3. Puerto 80 libre en el host (Apache/httpd detenido y deshabilitado).
4. Acceso SSH al servidor habilitado (sshd corriendo).
5. FirewallD activo para aplicar reglas de bloqueo externo.

2.2 Instrucciones de Ejecución

Despliegue inicial

Desde el directorio raíz del proyecto (donde reside el docker-compose.yml):

```
# 1. Bloquear puerto de pgAdmin desde el exterior
sudo firewall-cmd --permanent \
  --add-rich-rule='rule family="ipv4" port port="5050" protocol="tcp" reject'
sudo firewall-cmd --reload

# 2. Construir y levantar todos los servicios
docker compose up -d

# 3. Verificar que los 4 contenedores están corriendo
docker compose ps

# 4. Ejecutar protocolo de validación
bash scripts/validar.sh
```

Detener la infraestructura

```
docker compose down
```

2.3 Flujo de Interacción

Este proyecto no requiere entrada interactiva del usuario durante la ejecución. Todos los parámetros se configuran previamente en el archivo .env antes de levantar los servicios:

Variable	Valor por defecto	Descripción
POSTGRES_DB	reprobados_db	Nombre de la base de datos
POSTGRES_USER	db_admin	Usuario de PostgreSQL
POSTGRES_PASSWORD	Practica11.Segura!	Contraseña de PostgreSQL

Variable	Valor por defecto	Descripción
PGADMIN_EMAIL	admin@reprobados.com	Email de acceso a pgAdmin
PGADMIN_PASSWORD	PgAdmin.11!	Contraseña de pgAdmin
NGINX_PORT	80	Puerto público del balanceador
PGADMIN_PORT	5050	Puerto de pgAdmin (solo localhost)

3. Bitácora de Desarrollo y Configuración

3.1 Archivo docker-compose.yml

El archivo de orquestación define cuatro servicios, dos redes y dos volúmenes. Las decisiones de diseño más relevantes son:

- Credenciales externalizadas: todas las contraseñas y configuraciones sensibles se leen del archivo .env mediante la sintaxis \${VARIABLE}, eliminando valores hardcoded del archivo de orquestación.
- Binding de pgAdmin a 127.0.0.1: el puerto 5050 se mapea exclusivamente a la interfaz loopback del host (127.0.0.1:5050:80), haciéndolo invisible para conexiones externas.
- depends_on con condition: service_healthy: garantiza que pgAdmin no intente conectarse a la base de datos hasta que PostgreSQL haya pasado su healthcheck.
- restart: always en todos los servicios: los contenedores se recuperan automáticamente ante fallos del proceso principal.

3.2 Configuración de Nginx (nginx.conf)

La configuración de Nginx cumple dos roles simultáneos: balanceador de carga y hardening de seguridad. Los elementos principales son:

- server_tokens off: elimina la cabecera Server: nginx/X.X.X de todas las respuestas HTTP, ocultando la versión del servidor ante posibles atacantes.
- resolver 127.0.0.1 valid=30s: utiliza el DNS interno de Docker para resolver nombres de contenedores en tiempo de ejecución. Esto evita el error host not found in upstream al arrancar antes de que los contenedores dependientes estén listos.
- set \$upstream: combinado con el resolver, fuerza la resolución DNS por petición en lugar de por arranque, haciendo el proxy resiliente ante reinicios.
- location /app/: enruta peticiones al servidor de aplicaciones interno sin exponer su dirección IP ni puerto directamente al cliente.

3.3 Redes Multicapa

Se definieron dos redes Docker de tipo bridge con roles diferenciados:

Red	Contenedores	Propósito
red_publica	nginx_balanceador, app_server	Tráfico web externo. nginx expone el puerto 80.
red_datos	db_server, pgadmin	Aislamiento de datos. Sin acceso externo directo.

El aislamiento del plano de datos se refuerza a nivel de firewall: la regla rich-rule de firewalld rechaza conexiones entrantes al puerto 5050 desde cualquier IP que no sea localhost.

4. Pruebas de Funcionamiento (Validación)

4.1 Protocolo de Pruebas

Prueba	Acción	Salida Esperada	Salida Obtenida
11.1 Aislamiento de red	curl http://192.168.116.128:5050 desde el servidor	Connection refused — servicio invisible externamente	Connection refused. Puerto 5050 solo escucha en 127.0.0.1
11.2 DNS interno	docker exec nginx_balanceador wget - spider http://app_server/	Resolución exitosa por nombre de contenedor	PASS — nginx_balanceador alcanza app_server por DNS interno
11.3 Túnel SSH	ssh -L 8080:127.0.0.1:5050 shadou@192.168.116.128; abrir localhost:8080	Panel pgAdmin carga correctamente en el navegador	PASS — pgAdmin accesible vía túnel. Login exitoso con admin@reprobados.com
11.4 Persistencia	docker compose down && docker compose up -d; verificar datos	pgAdmin espera a db healthy; datos previos intactos en volumen	PASS — db_server healthy, 9 registros persistidos en db_data

4.2 Evidencias de Funcionamiento

docker compose ps — Estado de los contenedores

```
shadou@Srv-Linux-Sistemas:~/Administracion-de-Sistemas/scripts/tarea11$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
app_server	nginx:alpine	"/docker-entrypoint..."	app_server	3 minutes ago	Up 3 minutes (healthy)
db_server	postgres:16-alpine	"docker-entrypoint.s..."	db_server	3 minutes ago	Up 3 minutes (healthy)
nginx_balanceador	nginx:alpine	"/docker-entrypoint..."	nginx	3 minutes ago	Up 3 minutes (healthy)
pgadmin	dpage/pgadmin4:latest	"/entrypoint.sh"	pgadmin	3 minutes ago	Up 3 minutes

Prueba 11.1 — Aislamiento de red

```
shadou@Srv-Linux-Sistemas:~/Administracion-de-Sistemas/scripts/tarea11$ ss -tlnp | grep -E '80|5050|5432'
```

```
curl -v http://192.168.116.128:5050 2>&1 | head -10 # debe rechazar
```

```
LISTEN 0      4096      127.0.0.1:5050      0.0.0.0:*
LISTEN 0      4096      0.0.0.0:80         0.0.0.0:*
LISTEN 0      100      *:8080             *:*
```

```
LISTEN 0      1        [::ffff:127.0.0.1]:8005      *:*
```

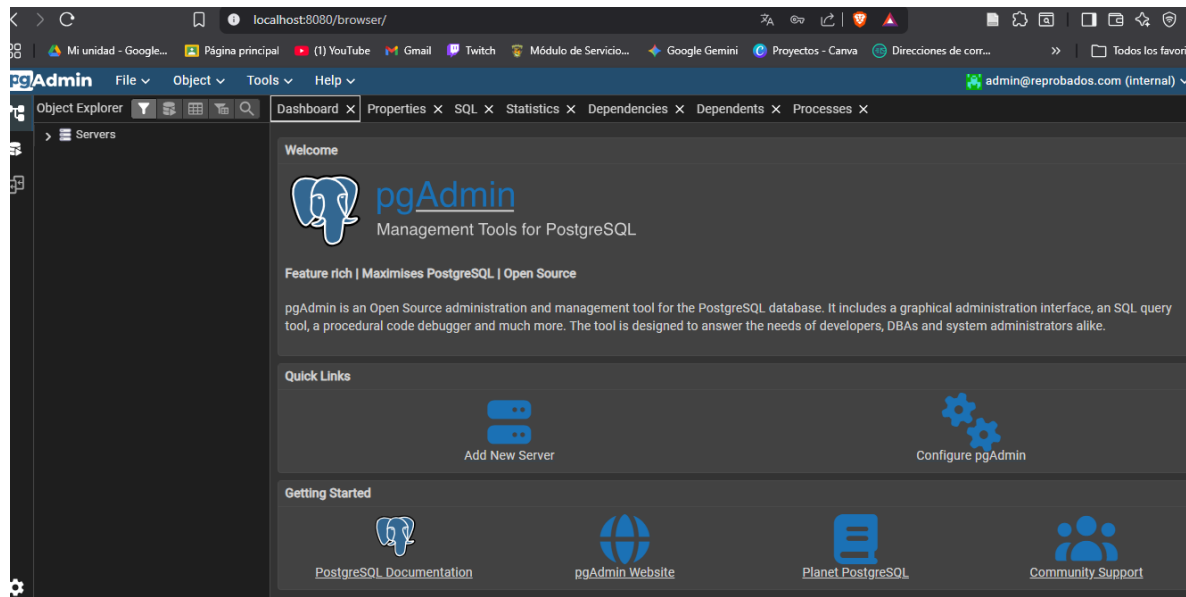
```
LISTEN 0      4096      [::]:80            [::]:*
```

Prueba 11.2 — Resolución DNS Interna

```
{
  "Name": "red_publica",
  "Id": "53c74057ab6dd0237497e628be0eac223b1bb25aa6538bd5ffa7fe8d09b269",
  "Created": "2026-05-10T13:11:04.073567206-07:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv4": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": null,
    "Config": [
      {
        "Subnet": "172.18.0.0/16",
        "Gateway": "172.18.0.1"
      }
    ]
  }
}
```

```
{
  "Name": "red_datos",
  "Id": "88cc05be2ec7372233c4f8b91fc9ad5bddff550313e6adf3cdcfecc7da23b296",
  "Created": "2026-05-10T13:11:04.192438693-07:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv4": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": null,
    "Config": [
      {
        "Subnet": "172.19.0.0/16",
        "Gateway": "172.19.0.1"
      }
    ]
  }
}
```

Prueba 11.3 — Túnel SSH hacia pgAdmin



Prueba 11.4 — Persistencia y Healthcheck

```
shadou@Srv-Linux-Sistemas:~/Administracion-de-Sistemas/scripts/tarea11$ docker inspect db_server --format '{{.State.Health.Status}}'
```

healthy

```
shadou@Srv-Linux-Sistemas:~/Administracion-de-Sistemas/scripts/tarea11$ docker volume ls
```

DRIVER	VOLUME NAME
local	09b2803b558bd4282e754cf7228744d2d78bd29198a85b9d34f4e36d35bebfcc
local	d9e43e27db53519e731ec0f98b37663ba56360ce0be68d231730d6d060f1ad18
local	db_data
local	pgadmin_data
local	web_content